

به نام خدا

1404/11/13

ش.د: 😊

نام و نام خوانوادگی: mmd

آزمون پایانترم مبانی برنامه نویسی کامپیوتر علموص

استاد: طیبه رفیعی

راه های ارتباطی:

1- @mmd1023 (تلگرام، بله و ...)

2- <https://github.com/mmd1023>

3- mmd.gh313@gmail.com

4- <https://t.me/ElectroGeek255>

دستورالعمل امتحان: توضیح 3 تا از سوالات تمرین های سری 2 تا 4 + نوشتن درسنامه برای 2 مبحث

تمرین سری 2

سوال 2 - تولد

برای پیدا کردن بهترین مخرج در هر مرحله از الگوریتم کسر مصری استفاده میکنیم.
این الگوریتم در واقع نوعی الگوریتم حریصانه است که در هر مرحله بزرگترین کسر $\frac{1}{n}$ که از $\frac{a}{b}$ کوچکتر باشد را پیدا میکند (با انتخاب حریصانه ی بزرگترین کسر کوچکتر از ورودی تضمین میشود که سریع به جواب میرسیم)؛ بنابراین چنین ثابت میشود:

$$\begin{aligned}\frac{1}{n} &< \frac{a}{b} \\ \frac{b}{a} &< n \\ n_{Best} &= \left\lceil \frac{b}{a} \right\rceil\end{aligned}$$

و از طرفی این نکته ریاضی را داریم:

$$\left\lceil \frac{x}{y} \right\rceil = \left\lfloor \frac{x+y-1}{y} \right\rfloor$$

بنابراین خواهیم داشت:

$$n_{Best} = \left\lfloor \frac{b+a-1}{a} \right\rfloor$$

تنها نکته باقی مانده برای گرفتن نمره کامل سوال این است که در سوال شرطی اضافه شده مبنی بر ممنوعیت $n = 1$ که وقتی کسر ورودی بزرگتر از 1 باشد بهترین انتخاب است. بنابراین برای رفع این مشکل تنها کافیسیت از یک متغیر کمکی به نام `healper`، چنین استفاده کنیم:

```

3  #include <iostream>
4  using namespace std;
5
6  int main()
7  {
8      long long a, b;
9      cin >> a >> b;
10
11     // ۲ = مخرج کمکی
12
13     while (a > 0)
14     {
15         long long x = (b + a - 1) / a;
16         // مخرج کمکی و مخرج یکم → x = ۱
17         // مخرج کمکی → x < ۱
18         if (x == 1)
19             cout << x << " ";
20         a = a * x - b;
21         b = b * x;
22         if (a == 0)
23             break;
24     }
25
26     return 0;
27 }

```

شرط تکرار نبوده

در انتهای هر مرحله هم کافیسیت با توجه به n بدست آمده، کسر $\frac{1}{n}$ را این چنین با مخرج مشترک گیری، از $\frac{a}{b}$ کم کنیم:

```

a = a * n - b;
b = b * n;

```

ورودی نمونه:

6 14

خروجی نمونه:

3 11 231

[کد نهایی نمره \(50/50\) \(برای باز شدن لینک گیتهاب کلیک کنید\)](#)

<https://github.com/mmd1023/BP/blob/main/ProblemSet2/2-tavalod.cpp>

تمرین سری 3

سوال 10 - تقسیم ریاضیاتی (امتیازی)

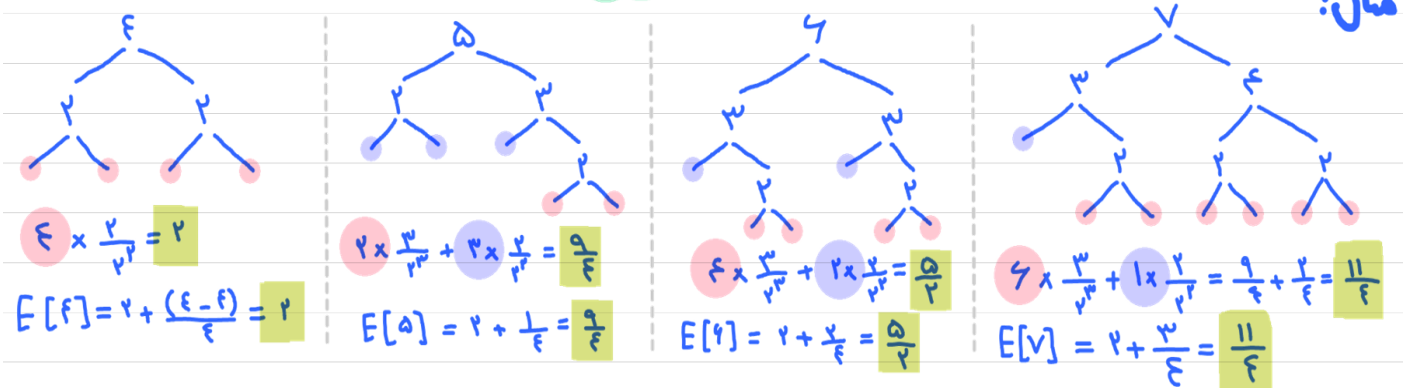
میتوان با استفاده از استقرای ریاضی فرمولی برای $E[x]$ پیدا کنیم:

فرض کنید n بزرگترین توان از 2 باشد که 2^n از x کوچکتر باشد، داریم:

$$n = \lfloor \log_2 x \rfloor$$

$$E[x] = \frac{n \cdot 2^n + (x - 2^n)}{2^n} = n + \frac{x - 2^n}{2^n}$$

مثال:



با توجه به بدیهی بودن فرمول پیدا شدن براساس مثال های اولیه و چند گام از استقرا، نیازی به بررسی دقیق و اثبات ریاضیاتی فرمول امید ریاضی وجود ندارد زیرا خارج از مبحث الگوریتم و برنامه نویسی است.

بنابراین فرمول محاسبه تابع $E[x]$ را به بهینه ترین روش ممکن (حتی بدون کد بازگشتی و بدون برنامه نویسی پویا) چنین بدست آمده است:

$$E[x] = \frac{n2^n + x - 2^n}{2^n}$$

توابع کمکی `mod_div` و `mod_pow`:

```
// a ^ b % mod
ll mod_pow(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
```

```

        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

```

تابع `mod_pow` یک تابع استاندارد است که به `fast_power` هم مشهور است، زیرا الگوریتم بهینه به توان رساندن و در عین حال باقی مانده ی اعداد توان دار را برخلاف مدل اولیه که پیچیدگی $O(b)$ دارد، صرفاً با $O(\log b)$ فراهم میکند. به این صورت که توان را هر دفعه دو قسمت میکنیم (اگر فرد بود یکی جدا میکنیم) و داریم: $a^{2x} = a^n \times a^x$ و یا $a^{2x+1} = a \times a^{2n} \times a^{2n}$

Tip 💡 :

```

b >> 1 = b / 2
b & 1 = b % 2

```

تابع `mod_div`:

```

// a / b % MOD
ll mod_div(ll a, ll b) {
    return (a % MOD) * mod_pow(b, MOD - 2, MOD) % MOD;
}

```

این تابع تقسیم پیمانه ای انجام می دهد: برای محاسبه ی $a/b \bmod MOD$ ، چون در حساب پیمانه ای تقسیم مستقیم نداریم، از **معکوس پیمانه ای** استفاده می کنه؛ اگر MOD عدد اول باشه، طبق قضیه ی فرما معکوس b برابر $b^{MOD-2} \bmod MOD$ هست، که با `mod_pow` حساب می شه، بعد هم در $a \% MOD$ ضرب و دوباره پیمانه می گیریم؛ بنابراین نتیجه معادل تقسیم a بر b در پیمانه ی MOD است.

تابع اصلی $E[x]$:

```
//  $E(x) = (n * 2^n + (x - 2^n)) / 2^n$ 
ll E(string x) {
    int n = x.size() - 1;

    ll x_mod = 0;
    for (char c : x)
        x_mod = (x_mod * 2 + (c - '0')) % MOD;

    ll pow_2_n = mod_pow(2, n, MOD);
    ll S = (((n % MOD) * pow_2_n) % MOD + x_mod - pow_2_n + MOD) % MOD;

    return mod_div(S, pow_2_n);
}
```

با توجه به باینری بودن رشته ورودی مقدار n به سادگی برابر میشود با یکی کمتر از طول رشته باینری:

```
int n = x.size() - 1;
```

تبدیل باینری به دسیمال:

```
ll x_mod = 0;
for (char c : x)
    x_mod = (x_mod * 2 + (c - '0')) % MOD;
```

با $(c - '0')$ در هر مرحله بزرگترین ارزش مکانی در مبنای دو ی باقی مانده در عدد را جدا کرده و با دو برابر کردن مقدار قبلی محاسبه شده تا اینجا و جمع این دو مقدار در نهایت عدد به مبنای دسیمال تبدیل میشود.

در نهایت فرمول را پیاده سازی میکنیم:

```
ll pow_2_n = mod_pow(2, n, MOD);
ll S = (((n % MOD) * pow_2_n) % MOD + x_mod - pow_2_n + MOD) % MOD;
```

توجه شود که در هر مرحله برای جلوگیری از سرریز کردن مقادیر $\% MOD$ میگیریم تا اعداد در بازه ایمن حفظ شوند.

ورودی نمونه:

3

3

110

3

100

10

1101001011

خروجی نمونه:

500000006

2

193359386

کد نهایی نمره (100/100) ☒ (برای بازشدن لینک گیتهاب کلیک کنید)

https://github.com/mmd1023/BP/blob/main/ProblemSet3/10-taghsim_riaziati.cpp

تمرین سری 4

سوال 2 - راه سکه ها

ایده بازگشتی سوال که بدیهتا به این صورت است که در هر مرحله، روی کل سکه ها حلقه زده میشه، و هر بار برنامه به شاخه ها مختلف منشعب میشه، اینطور که در یه شاخه از سکه i ام استفاده بشه (و این شاخه ادامه پیدا میکنه و دوباره ممکنه توش، همون سکه یا سکه های دیگه انتخاب بشن) و در شاخه های دیگه هم از سکه های دیگه در اون مرحله استفاده میشه. انتهای هر شاخه هم تعداد انتخاب ها محاسبه میشه و توی بازگشت ها، مینیمم این تعداد خروجی داده میشه.

مشکل این ایده اینه که تایم کاپلکسیته این الگوریتم نمایی هست و اگه ورودی ها یکم بزرگ بشه، تایم لیمت میخوریم! 🤖🤖

برای رفع این مشکل باید از دوباره محاسبه کردن شاخه های محاسبه شده اجتناب کنیم! (با تکنیک memoization که جواب های محاسبه شده را ذخیره میکند)، مسئله کلیدی در الگوریتم های بازگشتی اینه که بتونیم با memoization، زمان اجرا رو از نمایی به خطی یا چند جمله ای برسونیم 🙌

بنابراین کد اصلی تابع بازگشتی چنین میشود:

```
const long long INF = 1000000000000000000LL;
long long memo[100000];

long long solve(long long c, long long n, long long *coins) {
    if (c == 0) return 0;
    if (c < 0) return INF;

    if (memo[c] != -1) return memo[c];

    long long ans = INF;
    for (int i = 0; i < n; i++)
        ans = min(ans, solve(c - coins[i], n, coins) + 1);

    return memo[c] = ans;
}
```

آرایه memo جواب شاخه های محاسبه شده (کمترین تعداد سکه در با پایان هر شاخه) را با توجه به پول لازم یعنی c ذخیره میکند تا از دوباره کاری و نرخ نمایی جلوگیری شود.

شرط پایان یا base case هم در حالت هایی که مبلغ باقی مانده صفر شود (یعنی توانستیم با آن سکه ها آن مبلغ را فراهم کنیم) و حالتی که مبلغ منفی شود (یعنی نتوانسیم در آن شاخه به جواب برسیم) و همچنین حالتی که جواب شاخه را از قبل محاسبه کرده بودیم به نحو احسن هندل شده است.

و در گام ها بازگشت هم بدین صورت با کم کردن سکه از مبلغ و محاسبه ادامه شاخه و مینیم گیری، جواب ها را میسازیم:

```
for (int i = 0; i < n; i++)  
    ans = min(ans, solve(c - coins[i], n, coins) + 1);
```

فراخوانی در main:

در تابع main بعد از ورودی گرفتن هزینه و سکه ها، تابع را فراخوانی کرده، اگر جواب معتبر بدست آورد آن را و اگر جواب معتبر نبود 1- خروجی می‌دهیم:

```
long long res = solve(cost, n, coins);  
cout << ((res >= INF) ? -1 : res);
```

ورودی نمونه:

4

15 10 5 1

50

خروجی نمونه:

4

[کد نهایی نمره \(50/50\) \(برای بازشدن لینک گیت‌هاب کلیک کنید\)](https://github.com/mmd1023/BP/blob/main/ProblemSet4/2-rah_seke_ha_recursive_memo.cpp)

https://github.com/mmd1023/BP/blob/main/ProblemSet4/2-rah_seke_ha_recursive_memo.cpp

الگوریتم ها و توابع بازگشتی

💡 ایده‌ی بازگشتی اینطور است که برای حل کردن مسئله به ازای $f(n)$ ، فرض می‌کنیم که جواب مسئله را به ازای $f(n-1)$ رو داریم، بعد می‌خواهیم با استفاده از اون، $f(n)$ رو حساب کنیم. حالا وقتی برنامه اجرا میشه، میرود $f(n-1)$ ، بعد $f(n-2)$ و ... رو صدا می‌کنه تا برسه به حالت پایه (base)؛ مثلاً $f(0)$. حالت پایه هم همان شرطی است که باعث میشه برنامه توی recursion گیر نکند.

🤖 در واقع بازگشتی یه جورایی برعکس استقراست. توی استقرا اول حالت پایه رو می‌نویسیم که مسئله به ازای اون برقراره، بعد ثابت می‌کنیم اگر $f(k)$ برقرار باشه، پس $f(k+1)$ هم برقرار میشه.

یعنی رویکرد بازگشتی Top-Down هست، ولی استقرا و مخصوصاً توی Dynamic Programming معمولاً با نگاه Bottom-Up مسائل رو حل میکنن.

🔗 معمولاً ایده‌های بازگشتی زمانی میان وسط که برنامه حالت شاخه‌شاخه (منشعب) پیدا می‌کنه و عملاً سخت میشه فقط با حلقه مسئله رو نوشت (چون نمیدونیم چند تا حلقه می‌خوایم). هر بار تابع خودش رو به ازای حالت‌های جدید صدا می‌زنه و این شاخه‌ها همین‌طور ادامه پیدا می‌کنن تا برسن به حالت پایه. از اونجا به بعد هم مرحله‌به‌مرحله return می‌کنن به تابعی که اون‌ها رو صدا زده بود (parent call)، تا برگردن به همون $f(n)$ که از اول اجرا شده بود.

🔗 مثال تیپیکال recursion، محاسبه‌ی n مین عدد دنباله فیبوناچی:

```
int nthFibonacci(int n) {
    // Base case: if n is 0 or 1, return n
    if (n <= 1)
        return n;

    // Recursive case: sum of the two preceding Fibonacci numbers
    return nthFibonacci(n - 1) + nthFibonacci(n - 2);
}
```

مثال خروجی مشابه امتحان:

تابع به ازای 1

خروجی = 1

مثال های معروف دیگه که توشون از الگوریتم بازگشتی استفاده میشن:

Factorial ،Power ،GCD (الگوریتم اقلیدس) ،Merge Sort، generate all subsets ،Tower of Hanoi ،Recursive Binary Search ،DFS ،Preorder/Inorder/Postorder Traversal ،Permutations ،Combinations ،Recursive String Reverse ،Quick Sort ،Flood Fill ،Counting Paths in Grid ،Recursive Descent Parsing

! توجه !

در صورت تشابه درسنامه دیگران، حق کپی راییت این محتوای برای اینجانب بود و مدت ها پیش در چنل الکتروژیک در تلگرام این محتوا رو قرار داده بودم: ElectroGeek255@

لینک پست مربوط به الگوریتم بازگشتی: <https://t.me/ElectroGeek255/13>

شیء گرایی مقدماتی

برنامه نویسی شیء گرا (Object-Oriented Programming - OOP) درواقع یک طرز فکر برای طراحی و توسعه نرم افزار هاست. مثال دیگری از این طرز فکر ها functional programming هست.

درواقع OOP با الهام گیری از دنیای آبجکتیو پیرامونی ما ایجاد شده؛ به این صورت که میایم و مفاهیم را در قالب کلاس هایی برای کامپیوتر تعریف میکنیم و اشیا یا آبجکت هایی از نوع آن مفاهیم تعریف شده میسازیم، بدین ترتیب برنامه معنادار تر و خواناتر و قابل درک و قابل توسعه تر برای اپراتور انسانی میشود. به دیگر سخن OOP یک پارادایم (سبک فکری) در برنامه نویسی است که تلاش می کند دنیای واقعی را به صورت شیء (Object) مدل سازی کند. در این روش، به جای تمرکز صرف روی توابع و دستورات، تمرکز اصلی روی اشیاء، ویژگی ها و رفتارهای آن ها است.

یکی از مثال هایی که واقعا کاربردی است شیء گرایی برای مدیریت کاربران یک مجموعه است (مثال کلاس آدم و حیون خدایی مسخره هست که توی دوره ها میزنن بعضیا 😂)

چهار تا از مهم ترین مفاهیمی که باید در OOP یادگیریم:

1- کپسوله سازی (Encapsulation)

2- ارث بری (Inheritance)

3- چندریختی (Polymorphism)

4- انتزاع (Abstraction)

تعریف کلاس در C++ در ساده ترین فرم:

```
class User {
public:
    // Property
    string name;
    string username;
    string password;

    // Constructor
    User(string n, string un, string pass) {
        name = n;
        username = un;
        password = pass;
    }
}
```

```
// Destructor
~User() {}

// Behavior:
void login() {
    cout << username << " logged in\n";
}
void logout() {
    cout << username << " logged out\n";
}
};
```

کلاس با کلمه کلیدی class و اسم کلاس و {} تعریف میشود.

1- داخل کلاس ها متغیر هایی برای ذخیره ویژگی اشیا تعریف میشوند که به آنها Property میگوییم.

2- همچنین توابع داخل کلاس را که رفتار های اشیای آن نوع را تعیین میکنند را متود (method) گوییم.

تابع سازنده (Constructor):

تابع سازنده برای مقداردهی اولیه به ویژگی های شیء استفاده می شود. این تابع به طور خودکار هنگام ایجاد شیء فراخوانی می شود.

تابع مخرب (Destructor):

تابع مخرب معمولاً برای آزادسازی منابع استفاده می شود. این تابع به طور خودکار زمانی که شیء حذف می شود، فراخوانی می شود.

ساخت شی از کلاس و دسترسی به ویژگی های عمومی:

```
User user1("mmd", "mmd1023", "1234");
user1.name = "MMD";
user1.login();
cout << user1.name << " " << user1.username;
```

به سادگی میتوان فرض کرد که User یک دیتاتایپ جدید است که خودمان تعریف کردیم و شی ای از آن نوع میسازیم. همانطور که واضح است، میتوان با پاس دادن ویژگی های مورد نظر به تابع سازنده، پراپرتی ها را مقدار دهی اولیه کرد.

مثال خروجی مشابه امتحان برای کد بالا:

خروجی:

Mmd1023 logged in

MMD mmd1023

کپسوله‌سازی (Encapsulation):

```
class User {
private:
    string name;
    string username;
    string password;
public:
    // Getter methods
    string getName() { return name; }
    string getUsername() { return username; }

    // Setter methods with validation
    void setName(string n) {
        if(!n.empty()) name = n;
    }

    void setPassword(string pass) {
        if(pass.length() >= 8) {
            password = pass;
        } else {
            cout << "Password must be at least 8 characters";
        }
    }
};
```

کپسوله‌سازی از داده‌های حساس (مثل رمز عبور) محافظت می‌کند و با ارائه رابط‌های کنترل‌شده (getter/setter)، اعتبارسنجی داده‌ها و انعطاف‌پذیری در تغییر پیاده‌سازی داخلی را بدون تأثیر بر کدهای مصرف‌کننده امکان‌پذیر می‌سازد.

توجه: توضیح ارث بری و ... فراتر از هدف این درسنامه است و به علت ذیق وقت از آن صرف نظر میشود اما مثلاً در همین مثال آورده شده، میتوان کلاس های ادمین و مشتری و کارمند هم تعریف کرد که همگی از کلای کاربری ارث بری کنند!

مزایای OOP

- کد خواناتر و منظم تر
- توسعه و تغییر آسان
- کاهش تکرار
- تست پذیری بهتر
- مناسب برای تیم های بزرگ

معایب OOP

- پیچیدگی اولیه
- نیاز به طراحی دقیق
- سر بار بیشتر نسبت به کد ساده ی رویه ای

تبلیغات:

چنل جدید برای geek های تکنولوژی عه 🧐💧

شما به جمع دوستانه و مستقل خوره های تکنولوژی دعوت شده اید 🤖

<https://t.me/ElectroGeek255>

@ElectroGeek255

پرسپکتیو محتوایی: قراره از الگوریتم و دیتاستراکچر پایه شروع شه (کنار BP و AP بدرد بخوره 🤖)، بعدا یه دوری میزنیم توی پایتون و بقیه زبان ها 🙌، وقتی نیاز شد هم میریم سراغ متلب 📊، در این بین با آردوینو و در نهایت IOT که عشق خودمه هم آشنا میشیم 🤖 (مباحث دیگه مباحث طراحی مدار و ... هم الان یکم گنده گویی عه ولی چرا که نه)

@BP_with_mmd

@ElectroGeek255

@mmd1023